

```
+++  
date = '2026-04-05T20:53:07+08:00'  
title = 'App'  
+++
```

pthread_create() 创建线程的核心流程

1. 准备线程控制块（TCB）

- 内核分配线程控制块（Thread Control Block），保存线程的基本信息：
 - 线程 ID
 - 状态（就绪、运行、阻塞等）
 - 寄存器上下文（PC、SP 等）
- 如果传入了 `pthread_attr_t`，则根据属性设置栈大小、调度策略等。

2. 分配线程栈空间

- 每个线程有独立的栈，用于函数调用和局部变量。
- 默认大小由系统决定，可通过 `attr` 修改。

3. 初始化线程上下文

- 设置程序计数器（PC）指向入口函数 `start_routine`。
- 初始化栈指针（SP）和寄存器。
- 将参数 `arg` 传递给入口函数。

4. 加入调度器

- 内核把新线程放入 **可运行队列**。
- 根据调度策略（如 `SCHED_OTHER`、`SCHED_FIFO`、`SCHED_RR`）等待 CPU 调度。

5. 返回线程 ID

- `pthread_create()` 返回 0 表示成功。
- 用户态获得线程 ID，可用于 `pthread_join()` 或 `pthread_detach()`。

6. 线程开始执行

- CPU 调度器选择该线程运行。
- 执行入口函数 `start_routine(arg)`。
- 执行结束后调用 `pthread_exit()` 或隐式退出。

注意事项

- **共享地址空间**：所有线程共享进程的堆、全局变量。
- **独立栈**：每个线程有自己的栈和寄存器上下文。
- **资源释放**：如果不调用 `pthread_join()` 或 `pthread_detach()`，线程退出后资源不会自动释放，可能导致内存泄漏。

总结口诀

TCB → 栈 → 上下文 → 调度器 → 返回 ID → 执行入口函数

好的，我来帮你把 `pthread_join` 与 `pthread_detach` 的区别以及守护进程创建的所有步骤整理成“简短但展开”的形式，每一点都解释清楚“为什么”，但不啰嗦：

线程分离（Detach）与 join

`pthread_join`

- **阻塞等待**：调用线程会挂起，直到目标线程结束。这样保证调用方能在目标线程完成后继续执行，避免数据未准备好就使用。
- **获取返回值**：可以拿到目标线程的返回值，适合需要结果的场景，比如计算任务或状态反馈。
- **资源释放需手动**：线程结束后，资源不会自动释放，必须通过 `join` 来清理，否则会造成僵尸线程。

`pthread_detach`（线程分离）

- **非阻塞**：调用线程立即返回，不等待目标线程结束。这样主线程可以继续执行，不受子线程影响。
- **自动释放资源**：线程结束后系统自动清理，避免资源泄漏。
- **无法获取返回值**：因为线程已分离，调用方无法再 `join`，因此不能获取返回值。适合后台任务或日志处理等无需结果的场景。

join vs detach 总结

- **join**：同步、拿结果、手动释放。
- **detach**：异步、不要结果、自动释放。
- **选择原则**：需要结果就 `join`，不需要结果就 `detach`。

🔗 守护进程（Daemon）创建步骤

1. **fork 并退出父进程**
 - 父进程退出，子进程成为孤儿进程，由 `init` 收养。这样避免终端退出时子进程被杀死，保证后台独立运行。
2. **setsid 创建新会话**
 - 子进程成为新会话首进程，脱离原终端控制。这样不会收到 `SIGHUP` 信号，确保不受用户退出影响。
3. **改变工作目录**
 - 通常改为根目录 `/`，避免占用挂载点。否则守护进程可能阻止文件系统卸载。
4. **重设文件权限掩码 umask**
 - 设置为 `0`，确保守护进程创建的文件权限符合预期，而不是继承父进程的限制。
5. **关闭标准文件描述符**
 - 关闭 `stdin/stdout/stderr`，避免与终端交互。通常重定向到 `/dev/null`，保证后台运行时不会输出到终端。
6. **可选：重定向日志**
 - 将输出重定向到日志文件，如 `/var/log/daemon.log`。这样守护进程的运行信息可追踪，不依赖终端。

守护进程概念

- 守护进程：在后台运行的独立进程，不依赖终端
- 用途：提供系统服务（如 syslog、cron、nginx 等）

守护进程小结

- **核心思想**：脱离终端、后台独立、自动清理。
- **关键步骤**：fork → setsid → chdir → umask → 关闭文件描述符 → 日志重定向。
- **目的**：保证进程在后台长期稳定运行，不受用户登录/退出影响。

一句话总结：

join 是“等结果”，detach 是“自动清理”，daemon 是“后台独立”。

管道（pipe）与命名管道（FIFO）

1 管道（pipe）

- **定义**：一种进程间通信（IPC）机制，通过内核缓冲区在父子进程间传递数据。
- **特点**：
 - **半双工**：只能单向传输，若要双向需建立两条管道。
 - **匿名**：没有名字，只能用于有亲缘关系的进程（父子、兄弟）。这是因为内核只在进程继承关系中共享文件描述符。
 - **生命周期依赖进程**：进程结束时管道随之销毁。
- **创建方式**：int pipe(int fd[2]); 返回读端和写端文件描述符。
- **适用场景**：父子进程间简单通信，如 ls | grep txt。

2 命名管道（FIFO）

- **定义**：带名字的管道，存在于文件系统，可供无亲缘关系的进程通信。
- **特点**：
 - **半双工**：同样只能单向传输。
 - **有名字**：通过路径标识，进程可独立打开读写。
 - **生命周期独立**：只要文件存在，管道就存在，除非手动删除。
- **创建方式**：
 - mkfifo("/tmp/myfifo", 0666); 创建管道文件。
 - open("/tmp/myfifo", O_RDONLY / O_WRONLY); 打开读写端。
- **适用场景**：不同进程间通信，如后台服务与客户端交互。

3 区别

- **pipe**：匿名、仅限亲缘进程、随进程结束消失。
- **FIFO**：有名、可跨进程、存在于文件系统直到删除。

4 总结

- **匿名管道**：简单高效，适合父子进程通信。
- **命名管道**：灵活，可跨进程通信。

- **共同点：**都是半双工，遵循 FIFO 原则。

消息队列（Message Queue）

1 概念

- **定义：**一种 IPC 机制，用于在进程间传递固定格式的数据块。
- **特点：**
 - 支持异步通信，不要求进程同时存在。
 - 无亲缘关系进程也可通信。
 - FIFO 传递，但可带优先级。

2 POSIX 消息队列操作

1. **创建/打开：** `mq_open("/myqueue", O_CREAT|O_RDWR, 0666, NULL);`
2. **发送消息：** `mq_send(mq, "Hello", 5, 0);`
3. **接收消息：** `mq_receive(mq, buf, sizeof(buf), &prio);`
4. **关闭：** `mq_close(mq);`
5. **删除：** `mq_unlink("/myqueue");`

3 System V 消息队列操作

1. **创建/打开：** `msgget(key, IPC_CREAT|0666);`
2. **发送消息：** `msgsnd(msqid, &msg, sizeof(msg.mtext), 0);`
3. **接收消息：** `msgrcv(msqid, &msg, sizeof(msg.mtext), 1, 0);`
4. **删除：** `msgctl(msqid, IPC_RMID, NULL);`

4 使用注意事项

- **容量有限：**队列满时需处理阻塞或错误。
- **阻塞/非阻塞：**可选择等待或立即返回。
- **优先级支持：**POSIX 支持优先级，System V 仅按类型过滤。

5 总结

- **消息队列：**适合无亲缘关系进程的异步通信。
- **POSIX：**简单易用，支持优先级。
- **System V：**经典 IPC，兼容性好。
- **流程：**创建/打开 → 发送/接收 → 关闭 → 删除。

✦ 面试八股文总结：

- **pipe：**匿名、亲缘进程、随进程消失。
- **FIFO：**有名、跨进程、存在文件系统。
- **消息队列：**异步、无亲缘关系、支持优先级。

socketpair() 与管道的区别

1 基本概念

- 管道 (pipe)
 - 半双工通信：一次只能单向传输，若要双向需建立两条管道。
 - 亲缘关系限制：只能在父子或兄弟进程间使用，因为文件描述符只能通过 fork 继承。
 - 创建方式：pipe(fd[2])，返回读端和写端。
 - 生命周期依赖进程：匿名管道没有名字，进程结束后管道消失。
- socketpair()
 - 全双工通信：一对 UNIX 域套接字，支持双向传输。
 - 无亲缘关系也可用：只要能传递文件描述符（如通过 fork 或其他方式），不同进程即可通信。
 - 类似网络套接字：但不经过网络协议栈，效率更高。
 - 创建方式：socketpair(AF_UNIX, SOCK_STREAM, 0, sv);，返回两个可读写的套接字。

2 区别

- 通信方向：pipe 半双工，socketpair 全双工。
- 进程关系：pipe 仅限亲缘进程，socketpair 可跨进程。
- 接口特性：pipe 是文件描述符读写，socketpair 是套接字接口，支持更丰富的操作（如消息边界、select/poll/epoll）。
- 灵活性：socketpair 更接近网络编程模型，适合复杂 IPC；pipe 简单高效，适合轻量通信。

3 总结

- pipe：简单高效，半双工，适合父子进程通信。
- socketpair：全双工，可跨进程，支持更灵活的消息传递。
- 面试常考点：半双工 vs 全双工、是否依赖父子关系、是否支持消息边界。

阻塞与非阻塞 Socket 的区别

1 阻塞 Socket

- 特性：调用如 read()、write()、connect() 时，如果操作无法立即完成，线程会阻塞直到完成或出错。
- 优点：编程简单直观，CPU 不会忙等，操作系统负责挂起线程。
- 缺点：在网络延迟或对方未响应时，线程可能长时间阻塞，影响并发性能。
- 示例：

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
connect(sockfd, (struct sockaddr*)&addr, sizeof(addr)); // 阻塞直到连接成功或失败
read(sockfd, buf, sizeof(buf)); // 阻塞直到有数据
```

2 非阻塞 Socket

- **特性**：调用网络函数时，如果操作无法立即完成，函数立即返回，通常返回 -1 并设置 `errno = EAGAIN` 或 `EWOULDBLOCK`。
- **优点**：不会阻塞线程，适合高并发或异步 I/O。
- **缺点**：编程复杂度高，需要轮询或结合 I/O 多路复用（`select`、`poll`、`epoll`）。
- **示例**：

```
int flags = fcntl(sockfd, F_GETFL, 0);
fcntl(sockfd, F_SETFL, flags | O_NONBLOCK);

int ret = read(sockfd, buf, sizeof(buf));
if (ret < 0 && errno == EAGAIN) {
    // 暂无数据，稍后重试
}
```

3 区别总结

- **阻塞 Socket**：操作未完成时挂起线程，编程简单但不适合高并发。
- **非阻塞 Socket**：操作未完成时立即返回，需配合 I/O 多路复用，适合高并发场景。

4 总结

- **阻塞 socket**：直观易用，但在高并发场景容易造成线程阻塞。
- **非阻塞 socket**：结合 `select/poll/epoll` 可实现高并发、高效异步通信。
- **面试常考点**：阻塞 vs 非阻塞、I/O 多路复用的应用场景。

✦ 面试速记：

- **pipe**：半双工、亲缘进程。
- **socketpair**：全双工、跨进程。
- **阻塞 socket**：简单但会挂起。
- **非阻塞 socket**：复杂但高并发。

好的，下面我把 **pipe / socketpair / 阻塞 socket / 非阻塞 socket** 四者的核心区别整理成一个对比表格，结构清晰，适合面试快速回答：

四者对比表

特性	pipe	socketpair	阻塞 socket	非阻塞 socket
通信方向	半双工（单向）	全双工（双向）	全双工（双向）	全双工（双向）

特性	pipe	socketpair	阻塞 socket	非阻塞 socket
进程关系	仅限亲缘进程（父子/兄弟）	可跨进程（通过文件描述符传递）	任意进程间（网络通信）	任意进程间（网络通信）
接口类型	文件描述符读写	UNIX 域套接字	网络套接字（TCP/UDP）	网络套接字（TCP/UDP）
是否依赖进程生命周期	匿名，随进程结束消失	独立存在，直到关闭	独立存在，直到关闭	独立存在，直到关闭
消息边界	无，字节流	支持消息边界（SOCK_DGRAM）	TCP 无边界，UDP 有边界	TCP 无边界，UDP 有边界
阻塞特性	默认阻塞读写	默认阻塞读写，可设非阻塞	调用阻塞直到完成	调用立即返回，需轮询或多路复用
适用场景	父子进程间简单通信	本机进程间双向通信，灵活	简单直观的网络编程	高并发、异步网络编程
优缺点	简单高效，但只能单向且亲缘限制	灵活双向，可跨进程，但稍复杂	编程简单，但易阻塞	编程复杂，但高并发性能好

🌟 面试速记：

- pipe → 半双工 + 亲缘进程
- socketpair → 全双工 + 跨进程
- 阻塞 socket → 简单直观，但会挂起
- 非阻塞 socket → 复杂编程，但高并发

select、poll、epoll 的区别与使用场景

1 基本概念

- select / poll / epoll 都是 Linux 提供的 I/O 多路复用机制，用于同时监听多个文件描述符（socket、pipe 等）的可读、可写或异常状态。
- 目标：解决单线程处理高并发连接的问题，提高资源利用率。

2 select

- 特点：
 - 使用固定大小的位数组保存 fd 集合，大小受限于 FD_SETSIZE（通常 1024）。
 - 每次调用都要重新设置 fd 集合。
 - 内核扫描整个集合，返回可用 fd。
- 性能：O(n)，fd 数量多时效率低。
- 适用场景：连接数较少（几十到几百），简单 I/O 多路复用。

3 poll

- 特点：
 - 使用 `struct pollfd` 数组传递 fd，突破了 `FD_SETSIZE` 限制。
 - 每次调用仍需传递整个数组，内核逐个扫描。
- 性能：O(n)，与 `select` 类似。
- 适用场景：比 `select` 支持更多 fd，但性能仍受 fd 数量影响。

4 epoll

- 特点：
 - Linux 特有，使用内核维护的事件表。
 - 支持 水平触发（LT） 和 边沿触发（ET）。
 - fd 注册到内核后，只需调用 `epoll_wait` 获取就绪事件。
- 性能：O(1)，只返回就绪的 fd，效率远高于 `select/poll`。
- 适用场景：大规模并发（上千到上万连接），高性能服务器。

5 总结

- **select**：简单，受限于 fd 数量，适合少量连接。
- **poll**：突破 fd 限制，但性能仍是 O(n)，适合中等规模。
- **epoll**：高效 O(1)，适合海量连接，高并发场景。
- 面试重点：
 - O(n) vs O(1) 的性能差异。
 - fd 数量限制。
 - epoll 的 LT 与 ET 模式。

对比表

特性	select	poll	epoll
fd 数量限制	有限制（FD_SETSIZE，通常 1024）	无限制	无限制
数据结构	位数组	<code>struct pollfd</code> 数组	内核事件表
性能复杂度	O(n)，逐个扫描	O(n)，逐个扫描	O(1)，只返回就绪 fd
触发模式	无	无	支持 LT / ET
编程复杂度	简单	中等	较复杂
适用场景	少量连接	中等规模连接	海量连接，高并发服务器

✦ 面试速记：

- **select**：简单但有限制。
- **poll**：突破限制但仍 $O(n)$ 。
- **epoll**：高效 $O(1)$ ，适合高并发。

LT（水平触发）与 ET（边沿触发）的区别

1 水平触发（LT）

- **特点**：只要文件描述符处于可读/可写状态，`epoll_wait` 就会持续返回该事件。
- **行为**：
 - 事件会反复触发，直到应用程序将数据完全读/写消耗掉。
 - 行为与 `select` / `poll` 类似。
- **优点**：编程简单，容错性高，即使一次没读完数据，下次仍会触发。
- **缺点**：可能频繁触发，导致 `epoll_wait` 被反复唤醒，增加系统调用开销。

2 边沿触发（ET）

- **特点**：只有在状态发生变化时才触发一次事件。
- **行为**：
 - 文件描述符从不可读变为可读时触发。
 - 如果事件触发后没有一次性读完数据，下次不会再次触发。
- **优点**：减少 `epoll_wait` 的频繁唤醒，适合高并发场景。
- **缺点**：编程复杂，需要结合 **非阻塞 I/O** 并在事件触发时循环读/写直到返回 `EAGAIN`。

3 总结

- **LT**：默认模式，容易使用，事件会重复触发，适合一般场景。
- **ET**：高性能模式，只在状态变化时触发，需要非阻塞 I/O + 循环读写，适合高并发服务器。
- **面试重点**：
 - LT vs ET 的触发机制差异。
 - ET 模式下必须循环读取直到 `EAGAIN`。
 - ET 在 CPU 使用效率和高并发场景下更优。

✦ 面试速记：

- **LT** → 简单、重复触发、适合普通场景。
- **ET** → 高效、只触发一次、必须非阻塞循环读写。